



SOLID

STUPID, QUESACO ?!

- C'est un acronyme qui décrit les mauvaises pratiques en POO
- Définition des lettres
 - **S**ingleton
 - **T**ight Coupling
 - **U**ntestability
 - **P**remature Optimization
 - **I**ndescriptive Naming
 - **D**uplication

SINGLETON

- Instance Unique
- Le singleton est un design pattern qui n'autorise qu'une seule instance d'une classe.
- DANGER DE COUPLAGE FORT de notre code à cette classe qui ressemble à un state global
- TRES CONTROVERSE

TIGHT COUPLING

- COUPLAGE FORT
- Désigne la dépendance étroite d'une classe envers une ou plusieurs autres classes.
- Pose de gros soucis lors des tests ou encore lors de la mise à jour de librairies etc ...
- Très important de ne pas tomber dans ce piège afin de pouvoir isoler son code métier de l'architecture

TIGHT COUPLING

```
class MyClass
{
    public function construct()
    {
        $this->connexion = new MyConnexion();
    }
}
```



```
class MyClass
{
    public function construct(Connexion $connexion)
    {
        $this->connexion = $connexion;
    }
}
```

UNTESTABILITY

- NON TESTABILITÉ
- Tester un code ne devrait pas être complexe
- Un code non testable est quasi systématiquement un code qui ne suit pas le concept précédent de **TIGHT COUPLING**

PREMATURE OPTIMIZATION

- OPTIMISATION PRÉMATURÉE
- **Premature optimization is the root of all evil**
- La bonne façon de penser une application n'est de se lancer tout de suite, tête baissée dans la plus complexe architecture possible. Est à prendre en compte, le contexte, l'application, la durée de vie du projet ...
- L'écriture d'un projet se fait par refactorisation perpétuelle
- Trop de complexité peut rendre le code ingérable et non maintenable
- /!\ Ne pas confondre, complexité objective et complexité d'apprentissage

INDESCRIPTIVE NAMING

- NOMMAGE INDÉCHIFFRABLE
- Le nom de chaque variable, chaque fonction, chaque classe doit être clair, évident, précis
- Ne pas faire d'abréviation, mais ne pas faire de phrase de plusieurs lignes non plus
- Il est normal de passer du temps, beaucoup de temps à choisir le JUSTE nom pour ses variables, classes, fonctions ...
- **DE GRÂCE, RESPECTER LES CONVENTIONS DE NOMMAGE**

DUPLICATION

- DON'T REPEAT YOURSELF : DRY
- Garder son code simple est primordial
- Chaque répétition de code doit se faire poser la question : comment puis-je créer une nouvelle classe, une nouvelle fonction ?
- Le processus DRY est un élément clé de la refactorisation continue

SOLID, QUESACO ?!

- C'est un acronyme qui décrit les **bonnes** pratiques en POO
- **Michael Feathers et Robert C. Martin (aka Uncle Bob)**
- Va nous permettre d'écrire du code maintenable, faiblement couplé et testable
- Définition des lettres
 - **S**INGLE RESPONSIBILITY PRINCIPLE
 - **O**PEN / CLOSED PRINCIPLE
 - **L**ISKOV SUBSTITUTION PRINCIPLE
 - **I**NTERFACE SEGREGATION PRINCIPLE
 - **D**EPENDENCY INVERSION PRINCIPLE

SINGLE RESPONSIBILITY PRINCIPLE

- PRINCIPE DE RESPONSABILITÉ UNIQUE
- Une classe ne doit avoir qu'une seule responsabilité, en d'autres termes, elle ne doit avoir qu'une seule et unique raison de "changer"

OPEN/CLOSE PRINCIPLE

- PRINCIPE OUVERT / FERMÉ
- Une classe devrait être OUVÉRTE à l'extension, et FERMÉE à la modification
- Pour ajouter une fonctionnalité, il faudra créer une nouvelle classe et non pas modifier celle que l'on aura créé
- A avoir en tête quand on se retrouve à écrire des if pour choisir des comportements / algorithmes ...

LISKOV SUBSTITUTION PRINCIPLE

- PRINCIPE DE SUBSTITUTION DE LISKOV
- Si la classe T a une dépendance de type S alors on doit pouvoir remplacer cette dépendance par tous types dérivés de S
- Un cas de violation serait de TROP redéfinir de code lors de l'héritage par exemple
- Cela implique de conserver le type des valeurs d'entrée et de retour des fonctions, et donc de TYPER son code

INTERFACE SEGREGATION PRINCIPLE

- PRINCIPE DE SÉGRÉGATION DES INTERFACES
- Il s'agit du principe de responsabilité unique appliquée aux interfaces
- Plusieurs interfaces sont plus pertinent qu'une interface énorme dont certaines méthodes seront implémentées vides car non utiles
- Tout en veillant à ne pas violer le SRP, il est important de se rappeler qu'une classe peut implémenter plusieurs interfaces

DEPENDENCY INVERSION PRINCIPLE

- PRINCIPES D'INVERSION DE DÉPENDANCE
- Il s'agit probablement du plus important des principes à retenir
- Les classes doivent dépendre d'abstraction et non d'implémentation
- VA PERMETTRE D'ISOLER NOTRE CODE MÉTIER DE NOTRE ARCHITECTURE

REPÉRER LES CODES SMELLS

- Une question plus importante pourrait être de savoir comment savoir si vous devez appliquer les principes SOLID à votre code ou si vous écrivez du code qui n'est pas SOLID.
- Voici une liste de «tell», montrant que votre code pourrait avoir besoin d'être refactorisé :
 - Beaucoup d'instructions "if" pour gérer différentes situations dans une méthode.
 - Beaucoup de code qui ne fait rien pour satisfaire la conception de l'interface.
 - Vous continuez à ouvrir la même classe pour changer le code.
 - Vous écrivez du code dans des classes qui n'ont vraiment rien à voir avec cette classe. Par exemple, placer des requêtes SQL dans une classe en dehors de la classe de connexion à la base de données.

CONCLUSION

- SOLID n'est pas **une méthodologie parfaite** et peut conduire à des applications complexe, trop, et parfois conduire à l'écriture de code pas forcément nécessaire. Utiliser SOLID signifie écrire plus de classes et créer plus d'interfaces !!
- Néanmoins, cela oblige à séparer les responsabilités, à penser à l'héritage, à éviter la répétition du code et à aborder soigneusement la création d'application. Penser à la façon dont les objets s'assemblent dans une application est, après tout, ce qu'est la programmation orientée objet.

ALLER PLUS LOIN ...

- <https://williamdurand.fr/2013/06/03/object-calisthenics/#1-only-one-level-of-indentation-per-method>